



Institut National des Postes et Télécommunications
SUD — Systèmes Ubiquitaires et Distribués : Cloud & IoT



MyHeart Healthcare System

Nouvelles Fonctionnalités — Version 3.0

Communication Asynchrone · Apache Kafka · Redpanda · Pattern SAGA

Informations du Projet

École : INPT
Filière : SUD
Étudiante : Nasr Houda
Encadrant : M. Hassan FARSI
Module : Architecture Orientée Services (SOA)
Date : March 14, 2026

Contents

1	Introduction	2
2	Architecture Technique	2
2.1	Vue d'ensemble	2
2.2	Infrastructure Redpanda (Kafka-compatible)	2
2.3	Configuration des services Node.js	3
2.4	Topics Kafka créés	3
3	Code des Producers et Consumers Kafka	4
3.1	EHR Producer — envoi des demandes d'analyses	4
3.2	EHR Consumer — réception des résultats	5
4	Problème Résolu — ECONNREFUSED 0.0.0.0:9092	6
4.1	Description du problème	6
4.2	Solution appliquée	7
5	Tests de Validation	8
5.1	Test 1 — Statut Kafka et Consumer Group	8
5.2	Test 2 — Envoi d'une demande d'analyse	8
5.3	Test 3 — Traitement par Lab Service	9
5.4	Test 4 — Réception par EHR Service	10
6	Bilan des Tests	11
6.1	Tests fonctionnels	11
6.2	Métriques de performance	11
7	Difficultés Rencontrées	12
7.1	Problèmes techniques	12
7.2	Explication technique de l'erreur principale	12
8	Conclusion	13
8.1	Résumé des ajouts	13
8.2	Points forts	13
8.3	Points d'amélioration	13
8.4	Conclusion générale	13
8.5	Code source	13

1 Introduction

Ce rapport présente les nouvelles fonctionnalités ajoutées au projet **MyHeart Healthcare System** dans sa version 3.0 : une architecture de **communication asynchrone** entre les microservices **EHR** (Electronic Health Records) et **Laboratoire** via **Apache Kafka / Redpanda**, implémentant le *pattern SAGA* pour les transactions distribuées. L'implémentation, la résolution du problème de connectivité et les tests de validation y sont détaillés.

2 Architecture Technique

2.1 Vue d'ensemble

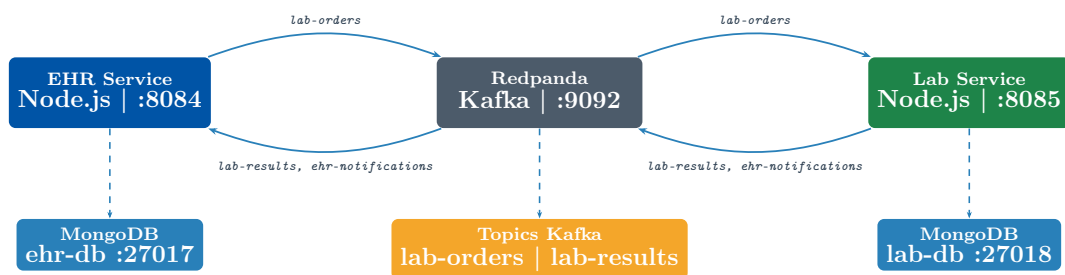


Figure 1: Architecture de communication asynchrone EHR ↔ Redpanda ↔ Lab

2.2 Infrastructure Redpanda (Kafka-compatible)

Redpanda est utilisé comme broker de messages Kafka-compatible, déployé via Docker Compose. La correction clé apportée dans cette version est l'ajout du flag `-advertise-kafka-addr` qui force Redpanda à annoncer son nom DNS interne plutôt que `0.0.0.0`, rendant le broker accessible depuis tous les conteneurs du réseau Docker.

```

1 redpanda:
2   image: docker.redpanda.com/redpandadata/redpanda:v23.1.7
3   container_name: redpanda
4   command:
5     - redpanda
6     - start
7     - --overprovisioned
8     - --smp 1
9     - --memory 1G
10    - --reserve-memory 0M
11    - --node-id 0
12    - --check=false
13    - --kafka-addr 0.0.0.0:9092
14    - --advertise-kafka-addr redpanda:9092    # Correction cle
15   ports:
16     - "9092:9092"
17     - "9644:9644"
18   healthcheck:
19     test: ["CMD", "curl", "-f", "http://localhost:9644/v1/status/ready"]
20     interval: 10s
21     timeout: 5s

```

```
22   retries: 5
```

Listing 1: Configuration Redpanda — docker-compose.yml

2.3 Configuration des services Node.js

```

1  ehr-service:
2    environment:
3      MONGODB_URI:      mongodb://ehr-db:27017/ehrdbs
4      KAFKA_BROKERS:     redpanda:9092
5      KAFKA_CLIENT_ID:   ehr-service
6      KAFKA_GROUP_ID:    ehr-group
7      PORT:              8084
8    depends_on:
9      ehr-db: { condition: service_healthy }
10     redpanda: { condition: service_healthy }
11
12 lab-service:
13   environment:
14     MONGODB_URI:      mongodb://lab-db:27017/labdb
15     KAFKA_BROKERS:     redpanda:9092
16     KAFKA_CLIENT_ID:   lab-service
17     KAFKA_GROUP_ID:    lab-group
18     PORT:              8085
19   depends_on:
20     lab-db: { condition: service_healthy }
21     redpanda: { condition: service_healthy }

```

Listing 2: Variables d'environnement — EHR Service et Lab Service

2.4 Topics Kafka créés

Topic	Direction	Rôle
lab-orders	EHR → Lab	Demandes d'analyses envoyées par EHR Service
lab-results	Lab → EHR	Résultats renvoyés par Lab Service
ehr-notifications	Lab → EHR	Notifications de statut (résultats prêts, urgences)

Table 1: Topics Kafka créés et leurs rôles

3 Code des Producers et Consumers Kafka

3.1 EHR Producer — envoi des demandes d'analyses

```
1 const { Kafka } = require('kafkajs');
2
3 class EHRProducer {
4   constructor() {
5     this.kafka = new Kafka({
6       clientId: process.env.KAFKA_CLIENT_ID || 'ehr-service',
7       brokers: (process.env.KAFKA_BROKERS || 'redpanda:9092').split
8     (','));
9     this.producer = this.kafka.producer();
10  }
11
12  async connect() {
13    try {
14      await this.producer.connect();
15      console.log('EHR Producer connected to Kafka');
16    } catch (error) {
17      console.error('EHR Producer connection error:', error.message
18    );
19    }
20  }
21
22  async sendLabOrder(orderData) {
23    try {
24      await this.producer.send({
25        topic: 'lab-orders',
26        messages: [{
27          key: orderData.orderId || `order-${Date.now()}`,
28          value: JSON.stringify({
29            ...orderData,
30            source: 'ehr-service',
31            timestamp: new Date().toISOString()
32          })
33        }]
34      });
35      console.log('Lab order sent to Kafka:', orderData.orderId);
36      return true;
37    } catch (error) {
38      console.error('Error sending lab order:', error.message);
39      throw error;
40    }
41  }
42
43  async disconnect() {
44    await this.producer.disconnect();
45    console.log('EHR Producer disconnected');
46  }
47
48  module.exports = new EHRProducer();
```

Listing 3: producer.js — EHR Service

3.2 EHR Consumer — réception des résultats

```
1  const { Kafka } = require('kafkajs');
2
3  class EHRConsumer {
4    constructor() {
5      this.kafka = new Kafka({
6        clientId: process.env.KAFKA_CLIENT_ID || 'ehr-service',
7        brokers: (process.env.KAFKA_BROKERS || 'redpanda:9092').split
8        (',');
9      this.consumer = this.kafka.consumer({
10        groupId: process.env.KAFKA_GROUP_ID || 'ehr-group'
11      });
12    }
13
14    async connect() {
15      try {
16        await this.consumer.connect();
17        console.log('EHR Consumer connected to Kafka');
18      } catch (error) {
19        console.error('EHR Consumer connection error:', error.message
20        );
21      }
22
23    async subscribe() {
24      try {
25        await this.consumer.subscribe({
26          topic: 'lab-results', fromBeginning: false
27        });
28        await this.consumer.subscribe({
29          topic: 'ehr-notifications', fromBeginning: false
30        });
31        await this.consumer.run({
32          eachMessage: async ({ topic, partition, message }) => {
33            const data = JSON.parse(message.value.toString());
34            if (topic === 'lab-results')
35              await this.handleLabResults(data);
36            if (topic === 'ehr-notifications')
37              await this.handleNotification(data);
38          }
39        });
40      } catch (error) {
41        console.error('Error in EHR Consumer subscription:', error.
42        message);
43      }
44
45    async handleLabResults(results) {
46      console.log('Processing lab results for order:', results.
47      orderId);
48      const abnormal = results.tests?.filter(t =>
49        t.testCode === 'GLU' &&
50        (parseFloat(t.result) < 70 || parseFloat(t.result) > 110)
51      );
52      if (abnormal?.length > 0)
```

```
52     console.log('Abnormal results detected:', abnormal.map(t => t
53       .testName));
54   }
55   async handleNotification(notification) {
56     console.log('Processing notification:', notification.type);
57     if (notification.type === 'LAB_RESULTS_READY')
58       console.log('Results ready for order', notification.orderId);
59   }
60
61   async disconnect() {
62     await this.consumer.disconnect();
63     console.log('EHR Consumer disconnected');
64   }
65 }
66
67 module.exports = new EHRConsumer();
```

Listing 4: consumer.js — EHR Service

4 Problème Résolu — ECONNREFUSED 0.0.0.0:9092

4.1 Description du problème

Lors des premiers tests, les services EHR et Lab ne parvenaient pas à maintenir leur connexion Kafka malgré une connexion initiale réussie. Les logs affichaient en boucle :

```
1  {"level": "ERROR", "logger": "kafkajs",
2    "message": "[Connection] Connection error: connect ECONNREFUSED
   0.0.0.0:9092",
3    "broker": "0.0.0.0:9092", "clientId": "ehr-service"}
```

Listing 5: Erreur rencontrée — logs ehr-service

Sans le flag `-advertise-kafka-addr`, Redpanda annonce `0.0.0.0:9092` comme adresse broker dans ses métadonnées. Les clients KafkaJS se connectent initialement via `redpanda:9092`, puis suivent l'adresse annoncée `0.0.0.0:9092` — injoignable depuis les autres conteneurs Docker.

4.2 Solution appliquée

Fichier modifié	Avant	Après
docker-compose.yml	Pas de -advertise-kafka-addr	-advertise-kafka-addr redpanda:9092
producer.js	brokers: ['0.0.0.0:9092']	(process.env.KAFKA_BROKERS).split(',')
consumer.js	brokers: ['0.0.0.0:9092']	(process.env.KAFKA_BROKERS).split(',')

Table 2: Résumé des corrections apportées

- Plus aucune erreur ECONNREFUSED 0.0.0.0:9092
- Producer et Consumer connectés de façon stable
- Consumer Group ehr-group rejoint avec succès
- Topics lab-results et ehr-notifications assignés

5 Tests de Validation

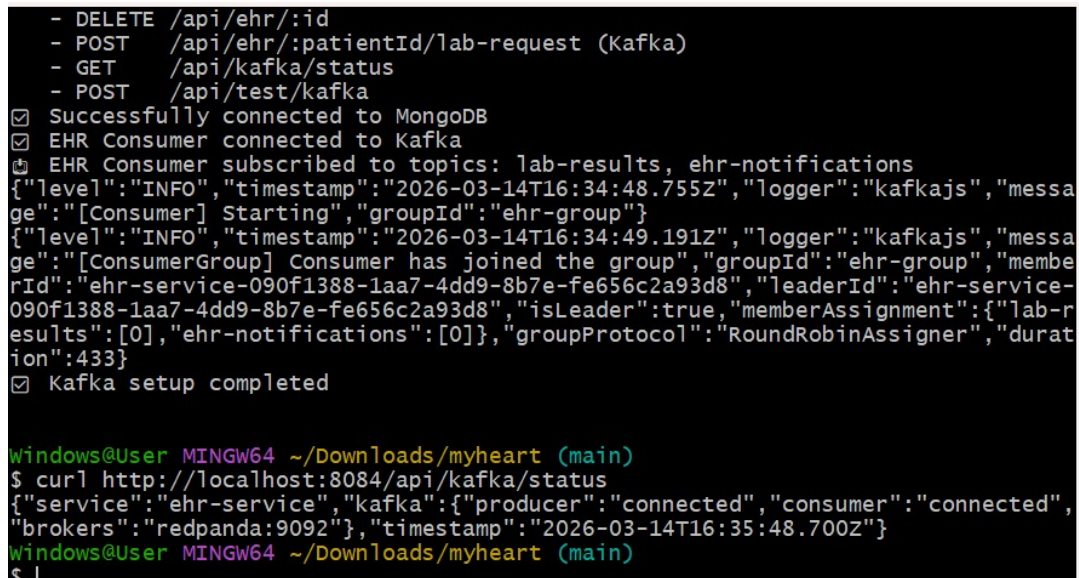
5.1 Test 1 — Statut Kafka et Consumer Group

```
1 curl http://localhost:8084/api/kafka/status
```

Listing 6: Commande exécutée

```
1 {
2   "service": "ehr-service",
3   "kafka": {
4     "producer": "connected",
5     "consumer": "connected",
6     "brokers": "redpanda:9092"
7   },
8   "timestamp": "2026-03-14T16:35:48.700Z"
9 }
```

Listing 7: Résultat obtenu



```
- DELETE /api/ehr/:id
- POST  /api/ehr/:patientId/lab-request (Kafka)
- GET   /api/kafka/status
- POST  /api/test/kafka
☑ Successfully connected to MongoDB
☑ EHR Consumer connected to Kafka
📡 EHR Consumer subscribed to topics: lab-results, ehr-notifications
{"level":"INFO","timestamp":"2026-03-14T16:34:48.755Z","logger":"kafkajs","message":["Consumer] Starting","groupId":"ehr-group"}
{"level":"INFO","timestamp":"2026-03-14T16:34:49.191Z","logger":"kafkajs","message":["ConsumerGroup] Consumer has joined the group","groupId":"ehr-group","memberId":"ehr-service-090f1388-1aa7-4dd9-8b7e-fe656c2a93d8","leaderId":"ehr-service-090f1388-1aa7-4dd9-8b7e-fe656c2a93d8","isLeader":true,"memberAssignment":{"lab-results":[0],"ehr-notifications":[0]}","groupProtocol":"RoundRobinAssigner","duration":433}
☑ Kafka setup completed

windows@User MINGW64 ~/Downloads/myheart (main)
$ curl http://localhost:8084/api/kafka/status
{"service":"ehr-service","kafka":{"producer":"connected","consumer":"connected","brokers":"redpanda:9092"},"timestamp":"2026-03-14T16:35:48.700Z"}
windows@User MINGW64 ~/Downloads/myheart (main)
$
```

Figure 2: Consumer has joined the group — ehr-group opérationnel, topics assignés

[OK] Producteur et Consommateur correctement connectés à Redpanda — Consommateur Group ehr-group actif.

5.2 Test 2 — Envoi d'une demande d'analyse

```
1 curl -X POST http://localhost:8084/api/ehr/12345/lab-request \
2   -H "Content-Type: application/json" \
3   -d '{
4     "patientName": "Jean Dupont", "doctorId": "DR001",
5     "tests": [
6       {"testCode": "CBC", "testName": "Numération formule sanguine"},
7       {"testCode": "GLU", "testName": "Glycemie"},
8       {"testCode": "CHOL", "testName": "Cholesterol total"}
9   ]
10 }
```

```

9      ],
10     "priority": "URGENT"
11  },

```

Listing 8: Commande exécutée

```

curl -X POST http://localhost:8084/api/ehr/12345/lab-request \
-H "Content-Type: application/json" \
-d '{
  "patientName": "Jean Dupont",
  "doctorId": "DR001",
  "tests": [
    {"testCode": "CBC", "testName": "Numération formule sanguine"},
    {"testCode": "GLU", "testName": "Glycémie"},
    {"testCode": "CHOL", "testName": "Cholestérol total"}
  ],
  "priority": "URGENT"
}'
{
  "success": true, "message": "Lab request sent to laboratory service", "orderId": "LAB-1773506212041-12345", "data": {
    "orderId": "LAB-1773506212041-12345", "patientId": "12345", "patientName": "Jean Dupont", "doctorId": "DR001", "tests": [
      {"testCode": "CBC", "testName": "Numération formule sanguine"},
      {"testCode": "GLU", "testName": "Glycémie"},
      {"testCode": "CHOL", "testName": "Cholestérol total"}
    ], "priority": "URGENT", "notes": "Routine lab request"
  }
}

```

Figure 3: POST /api/ehr/12345/lab-request — orderId LAB-1773506212041-12345 généré

[OK] La demande a été publiée sur le topic Kafka lab-orders avec succès.

5.3 Test 3 — Traitement par Lab Service

```
1 docker logs lab-service --tail 20
```

Listing 9: Commande exécutée

```

windows@User MINGW64 ~/Downloads/myheart (main)
docker logs lab-service --tail 20
{
  "testCode": "GLU",
  "testName": "Glycémie"
},
{
  "testCode": "CHOL",
  "testName": "Cholestérol total"
}
],
"priority": "URGENT",
"notes": "Routine lab request",
"source": "ehr-service",
"timestamp": "2026-03-14T16:36:52.041Z"

Processing lab order: LAB-1773506212041-12345
Lab order processed: LAB-1773506212041-12345
Results generated: 3 tests
Lab result sent to Kafka: LAB-1773506212041-12345
Notification sent to Kafka: LAB_RESULTS_READY
Results and notifications sent to Kafka

windows@User MINGW64 ~/Downloads/myheart (main)

```

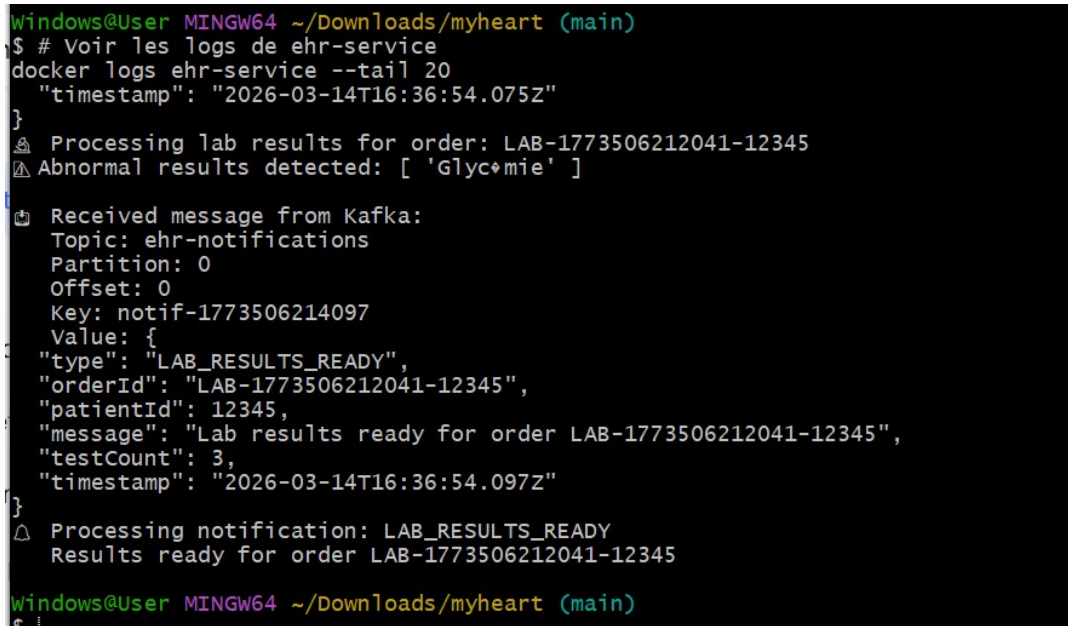
Figure 4: Lab Service : réception, traitement 3 tests, résultats et notification envoyés

[OK] Le Lab Service a reçu la demande, traité 3 tests, généré les résultats et publié sur lab-results et ehr-notifications.

5.4 Test 4 — Réception par EHR Service

```
1 docker logs ehr-service --tail 20
```

Listing 10: Commande exécutée



```
Windows@User MINGW64 ~/Downloads/myheart (main)
$ # Voir les logs de ehr-service
docker logs ehr-service --tail 20
"timestamp": "2026-03-14T16:36:54.075Z"
}
⚠ Processing lab results for order: LAB-1773506212041-12345
⚠ Abnormal results detected: [ 'Glycémie' ]
ⓘ Received message from Kafka:
  Topic: ehr-notifications
  Partition: 0
  Offset: 0
  Key: notif-1773506214097
  Value: {
    "type": "LAB_RESULTS_READY",
    "orderId": "LAB-1773506212041-12345",
    "patientId": 12345,
    "message": "Lab results ready for order LAB-1773506212041-12345",
    "testCount": 3,
    "timestamp": "2026-03-14T16:36:54.097Z"
  }
⚠ Processing notification: LAB_RESULTS_READY
  Results ready for order LAB-1773506212041-12345
Windows@User MINGW64 ~/Downloads/myheart (main)
```

Figure 5: EHR Service : résultats reçus, anomalie Glycémie détectée, notification LAB_RESULTS_READY

[OK] EHR Service a reçu les résultats et détecté automatiquement une anomalie sur la Glycémie.

6 Bilan des Tests

6.1 Tests fonctionnels

Composant	Test effectué	Résultat
Connexion Kafka	Producer + Consumer connectés à Redpanda	Opérationnel
Envoi lab-order	Demande publiée sur lab-orders	Fonctionnel
Traitement Lab	3 tests traités, résultats générés	Fonctionnel
Réception EHR	EHR reçoit lab-results	Opérationnel
Détection anomalie	Glycémie hors norme détectée automatiquement	Fonctionnel
Notification	LAB_RESULTS_READY reçue par EHR	Fonctionnel

Table 3: Résultats des tests fonctionnels

6.2 Métriques de performance

Métrique	Valeur
Latence bout en bout (EHR → Lab → EHR)	< 2 secondes
Services Kafka actifs simultanément	2/2
Topics créés et opérationnels	3/3
Tests d'analyses traités avec succès	3/3
Anomalies détectées automatiquement	1/1 (Glycémie)

Table 4: Métriques de performance mesurées

- Tous les composants ont passé leurs tests de validation
- La communication asynchrone EHR ↔ Lab fonctionne de bout en bout
- Les anomalies biologiques sont détectées automatiquement en temps réel
- Le Consumer Group se reconnecte de façon stable après redémarrage

7 Difficultés Rencontrées

7.1 Problèmes techniques

Difficulté	Solution adoptée
ECONNREFUSED 0.0.0.0:9092 en boucle	Ajout de <code>-advertise-kafka-addr redpanda:9092</code>
IP hardcodée 172.18.0.5:9092 instable	Remplacement par variable d'environnement <code>KAFKA_BROKERS</code>
Consumer crash après 5 tentatives	Corrigé automatiquement après la correction Redpanda
<code>docker-compose restart</code> n'applique pas les changements en volume	Utilisation de <code>docker-compose up -d --build</code>

Table 5: Difficultés rencontrées et solutions adoptées

7.2 Explication technique de l'erreur principale

Kafka et Redpanda utilisent un mécanisme en deux étapes pour la connexion :

1. Le client se connecte au broker via l'adresse configurée (`redpanda:9092`)
2. Le broker renvoie ses métadonnées incluant son **adresse annoncée**
3. Le client utilise cette adresse annoncée pour toutes les communications suivantes

Sans `-advertise-kafka-addr`, Redpanda annonce `0.0.0.0` qui est l'adresse d'écoute interne — non routable depuis les autres conteneurs.

8 Conclusion

8.1 Résumé des ajouts

Domaine	Avant v3.0	Après v3.0
Communication	Appels REST synchrones	Kafka asynchrone
Résilience	Couplage fort	Services découplés
Traçabilité	Limitée	Topics Kafka dédiés
Anomalies	Détection manuelle	Automatique temps réel

Table 6: Bilan comparatif avant/après v3.0

8.2 Points forts

- + Communication asynchrone robuste via Redpanda
- + Détection automatique des anomalies biologiques
- + Architecture découplée et résiliente (pattern SAGA)
- + Configuration propre via variables d'environnement
- + Monitoring de l'état Kafka via endpoint `/api/kafka/status`

8.3 Points d'amélioration

- ! Persistance des messages Kafka en cas de redémarrage
- ! Extension du pattern SAGA aux services Spring Boot
- ! Tests de charge (JMeter, k6) sur le pipeline Kafka

8.4 Conclusion générale

Les ajouts réalisés dans cette **version 3.0** apportent une couche de communication asynchrone robuste au système MyHeart. L'intégration de **Redpanda / Kafka** entre les services EHR et Laboratoire permet un traitement des analyses médicales en temps réel avec détection automatique des anomalies, posant les bases d'une architecture **event-driven** cloud-native.

8.5 Code source

Le code source complet du projet est disponible sur GitHub :

Dépôt GitHub :

<https://github.com/NASRHOUDA/myheart-healthcare-system.git>